ТЗ: Модуль совместной работы с телом документа

> **Статус: РЕАЛІЗОВАНО.** Це ТЗ модуля, написане перед розробкою; лишається як опис задуму й критеріїв приймання. Поточна поведінка — `docs/ARCHITECTURE.md` §9.
>
> Що додалось **понад** це ТЗ після його написання: заморожування документа на закритті голосування (`lockedAt` — read-only для всіх, включно з ADMIN, ADR-0006) і блокування прямого редагування тіла в статусі `IN_REVIEW`.

> Техническое задание для агента-разработчика на реализацию модуля **редактирования тела документа/стандарта** с тремя режимами (Просмотр / Предложения / Редактирование), потоком предложений и комментариев, прямым редактированием без согласования (по правам), закреплённой панелью WYSIWYG, автосохранением и предпросмотром изменений.
>
> Документ описывает **что и как** реализовать. Конкретные имена файлов в скобках — ориентир из эталонной реализации (Next.js App Router + tRPC + Prisma + TipTap), их можно адаптировать.

---

0. Контекст и стек

- **Назначение.** У сущности (стандарт или загруженный документ) есть «тело» — форматированный текст. С ним совместно работают несколько ролей: одни вносят правки напрямую, другие — только предлагают изменения и комментируют. Модуль обеспечивает оба сценария в одном UI.
- **Стек эталона:** Next.js 14 (App Router), tRPC, Prisma + PostgreSQL, TipTap (ProseMirror) для WYSIWYG, `mammoth` (импорт .docx → HTML), `docx` (экспорт Word), `@react-pdf/renderer` (экспорт PDF). Можно заменить аналогами, сохранив поведение.
- **Ключевые файлы эталона:** `src/components/StandardBodyEditor.tsx`, `src/components/ui/RichTextEditor.tsx`, `src/components/InlineComments.tsx`, `src/server/routers/suggestion.ts`, `src/lib/rbac.ts`, `src/lib/standardBody.ts`, `src/app/api/standards/[id]/import-body/route.ts`.

---

1. Роли и права

Модуль управляется ролевой матрицей. Три релевантных права:

Право	Что разрешает	Дефолт (роли)
`standard:editBody`	Прямое редактирование тела **без согласования** + принятие/отклонение предложений	Руководитель, заместитель, секретарь
`comment:add`	Вносить предложения (пропозиции) и inline-комментарии	Все участники
`standard:editMeta`	Редактирование карточки сущности (метаданных), **не тела**	Руководитель, заместитель, секретарь

Требования:

- **Матрица прав конфигурируется** в админке (тогглы «роль × действие»). Хардкод-дефолты лежат в коде (`rbac.ts`), но БД-оверрайды имеют приоритет.
- **Паритет клиент↔сервер.** Функция проверки `can(user, action, scope)` одна и та же на клиенте и сервере. Клиент получает те же оверрайды прав, что и сервер (отдельный запрос «мои оверрайды», бутстрап при загрузке). Сервер — единственный источник истины; UI лишь скрывает недоступное.
- **Глобальные роли** (ADMIN — всё; «директор центра» — видит всё, плюс настраиваемые права) применяются объединением (union) поверх роли в группе, без deny-правил.
- Изменение права применяется **без перелогина** (см. раздел про стабильность сессии в общих требованиях).

---

2. Модель данных

2.1 Тело сущности

- `bodyText` (для стандарта) / `bodyHtml` (для документа) — **HTML**, который эмитит WYSIWYG.
- `bodyUpdatedAt: DateTime?` , `bodyUpdatedById: String?` — кто и когда последним менял тело.
- Полиморфизм: предложения и комментарии могут относиться к стандарту **или** к документу (XOR на уровне приложения — заполнено ровно одно из `standardId` / `documentId`).

2.2 Предложение (`Suggestion`)

- `paragraphIndex: Int` — индекс блока в теле, к которому привязано предложение.
- `originalText: String` — снапшот HTML исходного блока на момент создания (для ре-анкеринга).
- `proposedText: String` — предлагаемый HTML (для DELETE — пусто).
- `operation: enum { REPLACE, INSERT_AFTER, DELETE }` .
- `status: enum { PENDING, ACCEPTED, REJECTED }` , `resolvedAt` , `resolvedById` , `resolveNote` .
- `rationale: String?` — обоснование автора.
- `author` (создатель), `createdAt` .
- `reactions: SuggestionReaction[]` — `{ userId, type: LIKE | DISLIKE }` (совещательные, без кворума).

2.3 Inline-комментарий (`InlineComment`)

- Якорь: `paragraphIndex: Int` + `startOffset: Int` + `endOffset: Int` + `selectionText: String` (снапшот выделенного текста — для ре-анкеринга при сдвиге текста).
- `status: enum { OPEN, RESOLVED }` , `resolvedById` , `resolvedAt` .
- Ветка ответов (reply chain), `author` , `createdAt` .

2.4 Аудит

- Каждая мутация тела/предложения пишет запись в журнал аудита: `action` (CREATE/UPDATE/DELETE/STATUS_CHANGE), `entity` , `entityId` , `before` / `after` (JSON), `note` , `userId` , `createdAt` . Проверка покрытия аудита — в CI (каждая мутация логирует или явно исключена).

---

3. Формат хранения тела и разбиение на блоки

Тело — HTML от WYSIWYG. **Блок** = top-level элемент (`<p>` , `<h1..3>` , `<ul>/<ol>` , `<blockquote>` , `<pre>` , `<table>`). Предложения и комментарии адресуются индексом блока.

3.1 Функция `splitBlocks(html): string[]`

- На клиенте (есть DOM): использовать `DOMParser` , брать `Array.from(root.children).map(el => el.outerHTML)` .
- На сервере / SSR (нет DOM): сканер по тегам с подсчётом глубины вложенности.
- **КРИТИЧНО — void-элементы.** При подсчёте глубины **не увеличивать** глубину для HTML void-элементов: `area, base, br, col, embed, hr, img, input, link, meta, param, source, track, wbr` . Иначе `<col>` внутри таблиц (которые эмитит WYSIWYG: `<table><colgroup><col><col>…`) «не закрывается», глубина не возвращается к 0, и **весь текст после первой таблицы слипается в один блок** → индексы параграфов «съезжают» → принятие предложений падает с ложным конфликтом «оригинальный текст не найден». Клиентский и серверный сплиттер обязаны давать **одинаковые границы блоков**.
- `joinBlocks(blocks): string` — конкатенация блоков.
- Нормализация: legacy plain-text (без тегов) мигрируется в `<p>` -обёртки прозрачно.

3.2 Сохранение стилей

- При импорте .docx сохранять: размеры шрифта (в **pt**), выравнивание, таблицы, ссылки, базовые начертания. Размер шрифта переносится через инлайновый `style="font-size: Npt"` (эталон: маппинг через стиль-имена mammoth → `span` с классом → пост-обработка в `pt`).

---

4. Три режима работы

Переключатель режимов — сегмент-контрол в шапке. Состояние режима в локальном стейте; **дефолт вычисляется лениво от прав** (чтобы не «прыгать», пока грузится сессия):

```
defaultMode = canEditBody ? 'edit' : canComment ? 'suggest' : 'view'
mode = pickedMode ?? defaultMode   // pickedMode = null, пока пользователь не переключил вручну
```

Доступность сегментов:

- **Просмотр** — всем;
- **Предложения** — у кого `comment:add`;
- **Редактирование** — только `standard:editBody`.

Если у пользователя только «Просмотр» (нет ни одного из прав) — переключатель не показывать.

Режим	Что показывает	Взаимодействие
Просмотр	Read-only статья по блокам	Подсветка существующих комментариев видна, но создавать нельзя; кнопок предложений/принятия нет
Предложения	Read-only статья по блокам + правая панель	Кнопки предложений на каждый блок; inline-комментарии через выделение; принятие/отклонение (для `editBody`); реакции
Редактирование	Всегда-включённый WYSIWYG + правая панель	Прямые правки с автосохранением; импорт .docx; правая панель показывает «Зміни/Коментарі» (список, без создания через выделение)

---

5. Режим «Редактирование» (WYSIWYG)

5.1 Редактор

- Всегда-включённый WYSIWYG (TipTap/ProseMirror). Расширения: базовое форматирование (bold/italic/ underline/strike), заголовки H1–H3, списки (ul/ol), цитата, код, **семейство и размер шрифта (размер в pt: 10–24)**, цвет текста, выравнивание (left/center/right/justify), ссылки, **таблицы**.
- Санитизация — строгой схемой редактора (принимаются только объявленные узлы/марки; инъекция скрипта невозможна по построению).
- Базовый шрифт тела — Times New Roman (чтобы импортированные доки выглядели «как в Word»); инлайновые стили перекрывают.

5.2 Автосохранение (без кнопки «Сохранить»)

- Debounce ~**1200 мс** после последнего ввода → мутация `updateBody`.
- **Flush при размонтировании** редактора (переключение режима, уход со страницы) — несохранённое не теряется.

- Сравнивать с последним сохранённым значением (через `ref`), чтобы не слать пустые сейвы.
- Статус в шапке: «Збереження…» (во время запроса) / «Збережено автоматично · дата · кто».
- **Контракт монтирования:** редактор сидится контентом **один раз** на `seedKey`. Рутинный поллинг (обновление тела другими) **не должен** перемонтировать редактор и затирать набираемый текст. Ключ `seedKey` бампится **только** при полной замене тела (импорт .docx). При бампе сбрасывать baseline и отменять отложенный автосейв (чтобы не гонять с импортной записью).

5.3 Закреплённая панель форматирования (привязка к шапке)

- Панель форматирования редактора — **sticky, прибита к верху** и не уезжает при скролле длинного документа.
- Над ней находится **sticky-шапка действий** (раздел 7). Панель форматирования прибивается **сразу под** шапкой действий: `toolbarTopOffset = высота шапки действий` (в эталоне ≈ 46px на странице / ≈ 117px в модалке = высота заголовка модалки + шапки действий). Две закреплённые полосы **стекаются**, а не перекрываются (см. 7.2).

5.4 Импорт .docx в режиме редактирования

- Кнопка «Імпортувати з Word» → загрузка файла → конвертация в HTML (server route, право `standard:editBody`).
- Тело уже существует → **подтверждение с предупреждением**, что будут удалены «осиротевшие» предложения/комментарии (их якоря указывают на старый текст). По подтверждению — `replaceBody` (атомарно заменяет тело и чистит предложения/inline-комментарии этой сущности).
- После импорта **перемонтировать редактор** на импортированный HTML (бамп `seedKey` + сид импортированным телом), чтобы то, что видит пользователь, совпадало с сохранённым. Оверрайд-сид сбрасывать при выходе из режима «Редактирование» (иначе при повторном входе затрёт правки, сделанные после импорта).

---

6. Режим «Предложения» (read-only + поток предложений и комментариев)

6.1 Статья по блокам

- Тело рендерится **поблочно** (read-only), каждый блок в обёртке с `data-paragraph-idx={idx}` (нужно для скролла к блоку и привязки выделения).
- На каждом блоке при наведении — мини-панель действий (для `comment:add`):
  - **Заменить** (REPLACE) — открыть редактор предложения, сидируя исходным HTML блока;
  - **Добавить после** (INSERT_AFTER);
  - **Удалить** (DELETE).
- Внизу — «Запропонувати новий параграф у кінці» (INSERT_AFTER к последнему блоку).

6.2 Inline-комментарии (через выделение)

- Overlay поверх статьи: пользователь **выделяет фрагмент текста** → всплывающий композер → создать комментарий, привязанный к `paragraphIndex + startOffset/endOffset + selectionText`.
- **Создание комментариев работает только на read-only статье** (в WYSIWYG нет такого overlay — у TipTap своя модель выделения). Поэтому комментарии «живут» в режиме «Предложения».
- В режиме «Просмотр» overlay смонтирован, но **создание выключено** (`canComment=false`) — видны только существующие подсветки.
- **Ре-анкеринг.** Если текст сместился, искать якорь по `selectionText`; если не найден — пометить комментарий «потерял якорь», не падать.

6.3 Правая панель «Зміни / Коментарі»

- Две вкладки с счётчиками (открытых/всего):
  - **Зміни** — список предложений (ожидающие + последние решённые), клик → скролл к блоку (подсветка).
  - **Коментарі** — список inline-комментариев, ответы, отметка «решено».
- Панель `sticky`, со своей внутренней прокруткой. Та же панель показывается и в режиме «Редактирование» (общий компонент), но **без overlay создания** (там нет read-only статьи).

6.4 Реакции

- На каждое предложение — 👍/👎 (совещательные, без порога/кворума), счётчики видны всем.

---

7. Шапка действий (sticky) и стекинг панелей

7.1 Состав шапки

- Слева: **переключатель режимов** (Просмотр/Предложения/Редагування) + статус (в edit — автосохранение; в read-only — «Оновлено …, N відкритих правок»).
- Справа: **Експортувати .docx**; в режиме редактирования ещё **Імпортувати з Word**.

7.2 Поведение sticky

- Шапка действий — `sticky` к верху (на странице `top: 0`; в модалке — под заголовком модалки, напр. `top: 71px` на desktop). `z-index` шапки **выше** панели форматирования.
- Панель форматирования редактора прибивается **сразу под** шапкой действий (её `top` = низ шапки). Полосы стекаются, без перекрытия и без «щели», сквозь которую виден прокручиваемый текст.

---

8. Принятие / отклонение предложений

8.1 Модалка-предпросмотр (перед применением)

- Клик «Прийняти» на предложении открывает **модалку подтверждения** с дифом того, что изменится:
  - REPLACE — пословный diff (red strike = удалено, green = добавлено) + серое исходное;
  - INSERT_AFTER — новый блок (зелёный);
  - DELETE — удаляемый блок (красный, зачёркнутый);
  - плюс автор и обоснование.
- Кнопки «Прийняти зміну» / «Скасувати». Диф вынести в общий компонент (карточка + модалка используют один).

8.2 Серверная логика `accept` (устойчивая к конкуренции)

Заново читать текущее тело и **ре-анкерить по содержимому**, а не слепо по индексу:

1. Взять блок по `paragraphIndex`. Сравнить plain-text (strip тегов + нормализация пробелов/энтити) с `originalText`. Если совпало — применить здесь.
2. Иначе — искать блок, чей plain-text равен `originalText`. Найден — применить по нему.
3. Если не найден:
   - DELETE — блок уже удалён → пометить решённым (no-op);
   - INSERT_AFTER — добавить `proposedText` в конец (намерение «добавить контент» не теряем);
   - REPLACE — это реальный конфликт → вернуть понятную ошибку «оригінальний текст параграфа №N вже не знайдено (хтось його змінив), перегляньте правку та створіть нову».

- Детект «уже применено»: если `proposedText` уже присутствует в каком-то блоке — пометить решённым, не дублировать.
- Применение: REPLACE — замена блока; DELETE — удаление; INSERT_AFTER — вставка после. Затем `join` блоков (выкинуть пустые), сохранить тело (`bodyUpdatedAt/By`), обновить статус предложения, записать аудит, уведомить автора.
- **Совпадение сплиттера клиент/сервер обязательно** (см. 3.1) — иначе ложные конфликты.

8.3 Модель конкурентности

- Last-write-wins для тела + ре-анкеринг для предложений. Кворум/блокировки не требуются. Сообщения о конфликте — понятные, без потери данных.

---

9. Серверный API (tRPC-ориентир)

| Процедура | Право | Назначение | | ---------- | ---------- | ---------- | | `suggestion.list({standardId | documentId})` | чтение | Список предложений (поллинг ~5с) | | `suggestion.create(target, paragraphIndex, originalText,`

`proposedText, operation, rationale?)` | `comment:add` | Создать предложение | |
`suggestion.react({suggestionId, type | null})` | `comment:add` | 👍/👎 / снять | |
`suggestion.accept({id, note?})` | `standard:editBody` | Принять (см. 8.2) | |
`suggestion.reject({id, note?})` | `standard:editBody` | Отклонить | |
`suggestion.updateBody(target, bodyText)` | `standard:editBody` | Прямое сохранение тела (автосейв) | | `suggestion.replaceBody(target, bodyText)` | `standard:editBody` | Атомарная замена тела + чистка осиротевших предложений/комментариев (импорт) | | `inlineComment.list / create / reply / resolve` | `comment:add` | Inline-комментарии | | `POST /api/.../import-body` | `standard:editBody` | Конвертация .docx → HTML |

Все мутации: проверка права на сервере (403 при отказе) + запись аудита.

---

10. Уведомления, экспорт, прочее

- **Уведомления (колокольчик):** «вам предложили правку», «ваша правка принята/отклонена», упоминания в комментариях. Текст обезличен корректно (без подстановки чужого имени).
- **Экспорт** тела в Word (.docx) и PDF с сохранением форматирования (шрифты, размеры, выравнивание, таблицы, ссылки); единый шрифт Times New Roman; статические шрифты для PDF.
- **Пустое тело:** для `editBody` — предложить «Імпортувати .docx» или «Почати з нуля»; для остальных — «Текст ще не додано».

---

11. Edge cases / нефункциональные

- Не перемонтировать WYSIWYG на рутинном поллинге (затрёт ввод) — только на импорте.
- Не ломать поток предложений/комментариев для пользователей **без** `editBody` (режимы Просмотр/ Предложения работают как раньше).
- Большие документы (десятки-сотни блоков, таблицы): сплиттер и рендер должны выдерживать; не монтировать per-параграф WYSIWYG-overlay (в эталоне это «вешало» рендер — отсюда раздельные режимы).
- Удаление документов/тел — обратимо через аудит/архив (физического удаления нет).

---

12. Критерии приёмки

1. Пользователь с `editBody` открывает тело → сразу режим «Редагування», WYSIWYG, автосохранение работает, статус обновляется, кнопки «Зберегти» нет.
2. Панель форматирования и шапка действий **закреплены** и не уезжают при скролле; не перекрываются.
3. Переключение Просмотр/Предложения/Редагування доступно по правам; дефолт не «прыгает» на загрузке.

4. В «Предложениях»: создаются предложения (заменить/добавить/удалить) и inline-комментарии через выделение; видна правая панель со счётчиками.
5. Принятие предложения открывает **модалку-предпросмотр с дифом**; по подтверждению правка применяется.
6. Принятие работает на документе **с таблицами** (нет ложного «параграф не знайдено»): клиентский и серверный сплиттер дают одинаковые блоки, void-`<col>` не ломает индексы.
7. Импорт .docx заменяет тело, чистит осиротевшие предложения, перемонтирует редактор на новое тело.
8. Пользователь **без** `editBody` видит Просмотр/Предложения, прямого редактирования и принятия нет.
9. Каждая мутация тела/предложения пишет аудит; экспорт Word/PDF сохраняет форматирование.

Документ описывает уже реализованный в проекте модуль (StandardBodyEditor и связанные роутеры) как переносимое ТЗ. Конкретные пиксельные offset'ы и тайминги — ориентировочные, подбираются под рендер.